

Lakeflow Declarative Pipelines



Lakeflow Declarative Pipelines

Data Pipeline workflow : Ingestion -> Transformations (ETL Activity)

Lakeflow connect is for Data Ingestion. After the data is ingested into the delta lake, the next step is to perform transformations. (The transformation is for converting the data from one form to another. This is an ETL activity)

With the introduction of **Lakeflow Declarative Pipelines by Databricks**, a lot of complex repetitive tasks performed by the Data Engineers are made easy.

This involves writing the code in a declarative approach. That is informing the system “What needs to be done” rather than “How it has to be done”.

DLT (Delta Live Tables) was rebranded and is now known as **Lakeflow Declarative Pipelines**.

There are a very few minor feature changes introduced in the Lakeflow Declarative Pipelines. One of them is the introduction of a new IDE, which makes the development much easier.

(The lakeflow pipeline editor feature can be turned off, if you want to continue with the previous conventional Notebook approach of developing the code.)

It is essential to know DLT for the reason that few companies are still yet to introduce the new Lakeflow Declarative Pipelines in their projects and this was a recent feature change / rebranding. Therefore, it is important to have an understanding of DLT as well.

Delta Live Tables (DLT)

Important Highlights :

- Databricks open sourced its declarative pipeline framework and contributed it to the Apache Spark open source project.
- Certain Networking options need to be configured during the databricks workspace creation to save cost.
- Azure Data Factory (or Lakeflow Connect) would be used for ingesting the data to Delta lake

- The data present in the storage account of Azure can be accessed and viewed in Databrick using mount point. Mount point is created to refer to the container present in the storage account.
- Serverless compute were not present earlier. We only had the option for classic compute and serverless SQL warehouse.



Building Declarative ETL Pipelines with DLT



What is DLT?

DLT is a framework to build ETL pipelines in a very simplified way using a declarative approach. Simplifies the ETL process.

What is Declarative Approach?

An approach where you tell the system what needs to be done and not how it has to be done. The system then uses the best way to get the task done.

Example :

Manually implementing SCD Type 2 / xMerge strategy would involve a lot of complexity and might go wrong if not done the right way.

It would be less prone to errors if it is handled by the system internally. And we just tell the system to implement SCD type 2 without saying how it has to be done. i.e., no code for implementing SCD is provided. The system itself does everything that is needed to get the task done.

DLT simplifies traditional ETL

What is ETL and what are the challenges in traditional ETL ?

In ETL, you need to take care of several things manually

- How to “Extract the Data”
- How to “Transform the Data”
- How to “Load the Data”
- Need to setup an orchestration framework manually
- Infrastructure Management
- Scaling up and Scaling Down the infrastructure
- Manage Dependencies on your own.
- Track the lineage (what is the up-stream, down-stream)
- Handling Batch and streaming in the pipeline.
- Ensure Data Quality
- Handling Data failure and retries
- Monitor and Optimize pipeline
- Deploy in multiple environments

And many such activities need to be handled and the time spent on tooling dominates your effort.

DLT solves the above mentioned challenges of traditional ETL and makes it easier to build ETL pipelines.

Advantage of Declarative Programming :

DLT will take care of all the repetitive setup and maintenance related tasks.

1. We write the declarative code (in SQL or Python)
2. We create a DLT pipeline to run the DLT code

Demo1

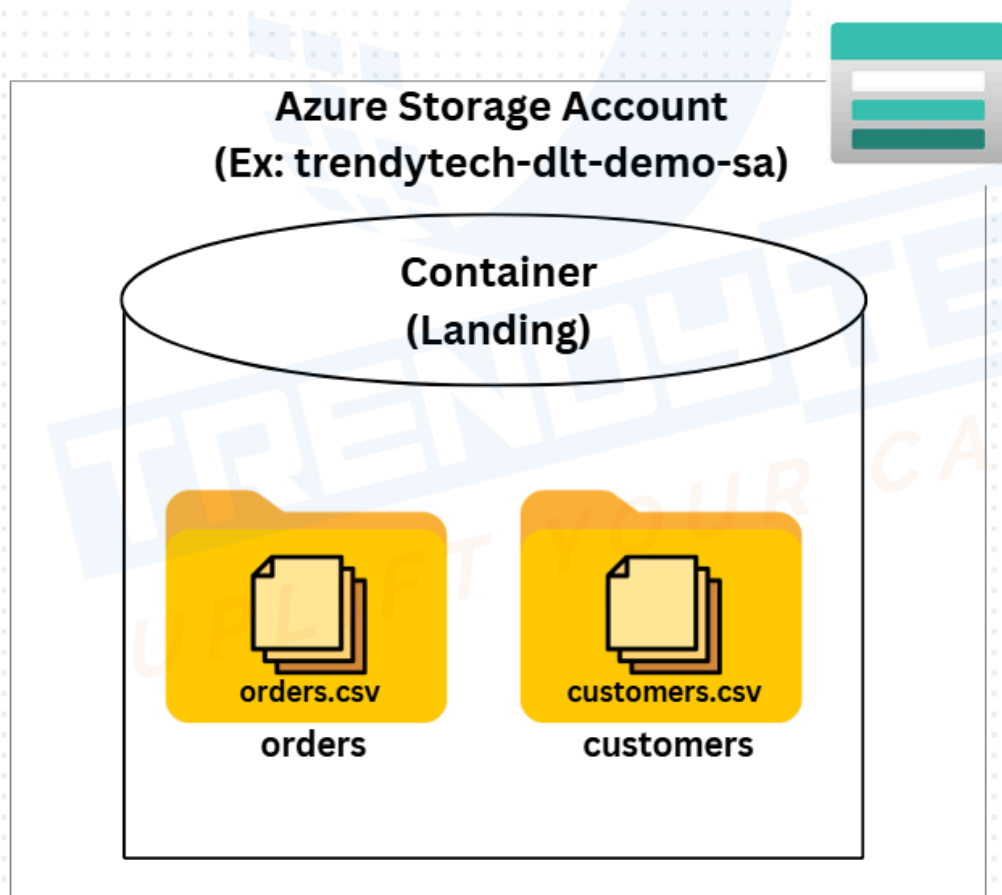
=====

Orders : Transactional Data

Customers data : Master Data

Medallion Architecture

Landing -> Bronze (RAW) -> Silver -> Gold



Landing - Contains raw data (in formats like CSV).

Every Layer serves different Personas.

Bronze - Tables will be created in the bronze layer. Data will be in the Delta format. Bronze will act like the source of truth. **(This layer is usually for the Data Engineers)**

Silver - This layer is for activities like : cleaning, enriching, merging the data and also for SCD2 implementation

1. Silver Cleaned - Implementing Data Quality Checks.
2. Silver Merged - Strategy to remove duplicates with the help of data merge or SCD2 implementation.
[for **orders** - Merge & **customers** - SCD2]

(This layer is for Downstream teams like - Machine Learning team / Data Scientists / Data Analysts)

Gold - Contains Domain specific data. Data is aggregated for reporting purposes.

(This layer is for Marketing / Sales teams)

Demo Steps :

1. Create Storage Account
2. Create Landing Container inside storage account
3. Create Orders and Customers Directories
4. Create Databricks Workspace
5. Create Mount Point to connect to the Databricks Workspace. (To refer to the Storage container)
6. Create a General Purpose Cluster with minimal resources.

Writing Declarative Code



Declarative approach of writing ETL code

Demo Steps :

1. Load the Data from the Landing Zone and move it to the Bronze Layer.

CREATE STREAMING TABLE orders_bronze (This creates a table in the bronze layer that loads the data in an **incremental way** and not a Full load each time the new data flows in. This is done with just using the keyword “STREAMING” whilst creating the table.

When the inferSchema is set to True, the column names and types are set to the values that are inferred from the schema.

```
CREATE STREAMING TABLE orders_bronze
as
SELECT *,
input_file_name() as filename,
current_timestamp() as load_time
FROM cloud_files('/mnt/landing/orders', 'csv', map("cloudFiles.inferColumnTypes","True"))

CREATE STREAMING TABLE customers_bronze
as
SELECT *,
input_file_name() as filename,
current_timestamp() as load_time
FROM cloud_files('/mnt/landing/customers', 'csv', map("cloudFiles.inferColumnTypes","True"));
```

2. The data from the Bronze layer is taken, cleaned and loaded into the **Silver Cleaned** layer. Specify the constraints while creating the table.

```
CREATE STREAMING TABLE orders_silver_cleaned (
    CONSTRAINT valid_order EXPECT (order_id is NOT NULL) ON VIOLATION DROP ROW,
    CONSTRAINT valid_customer EXPECT (customer_id is NOT NULL) ON VIOLATION DROP ROW
) as
SELECT orderid as order_id,
orderdate as order_date,
customerid as customer_id,
totalamount as total_amount,
status,
filename as file_name,
load_time
FROM STREAM(live.orders_bronze)
```

```
CREATE STREAMING TABLE customers_silver_cleaned(
  CONSTRAINT valid_customer EXPECT (customer_id is NOT NULL) ON VIOLATION DROP ROW
) as
SELECT customerid as customer_id,
customername as customer_name,
address as city,
dateofbirth as dob,
registrationdate as customer_since,
filename as file_name,
load_time
from STREAM(live.customers_bronze);
```

3. Data is taken from **Silver Cleaned** and the Duplicates are handled. Duplicates can be handled using Merge or SCD2. The data without duplicates is then loaded into the **Silver Merged** layer.

```
CREATE STREAMING TABLE customers_silver;

APPLY CHANGES into live.customers_silver
from STREAM(live.customers_silver_cleaned)
keys(customer_id)
sequence by load_time
stored as scd type 2;
```

4. Orders Data is taken from the Silver Cleaned. Duplicates are handled using **Merge** and the data is loaded to the **Silver Merged** layer.

```
CREATE STREAMING TABLE orders_silver;

APPLY CHANGES into live.orders_silver
from STREAM(live.orders_silver_cleaned)
keys(order_id)
sequence by load_time;
```

5. Data from the Silver Layer is taken to perform aggregation and then finally loaded to the **Gold Layer** where the downstream teams like Business or data analysts will use it to generate the reports.

CREATE MATERIALIZED VIEW - (~Full Load) creates the table with the complete data right from beginning unlike the STREAMING which takes only the newly arrived data.

```
CREATE MATERIALIZED VIEW city_wise_sales_gold
as
SELECT city, sum(total_amount) as total_sales
from live.orders_silver o
join live.customers_silver c
on o.customer_id = c.customer_id
group by city;
```

Execution using Hive Metastore



Execution Considering Hive Metastore

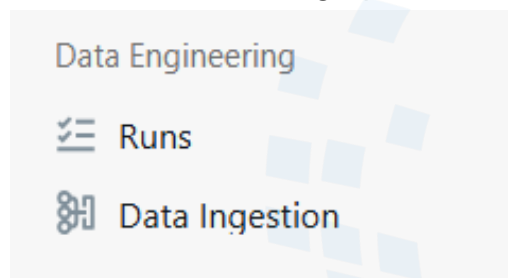
DLT cannot use Serverless with Hive Metastore. It will be creating its own compute.

In the Earlier Interface under the Data Engineering we had the following options:

Data Engineering

- Job Runs
- Data Ingestion
- Delta Live Tables

But now the following options are available :



The Services / resources that will be used in the upcoming sessions :

- Serverless SQL Warehouse (Wherever SQL queries needs to be executed)
- Job Compute Cluster (Physical servers are created on the go while the Job is being executed)
- Hive Metastore (To see the developments that has happened)
- Mount points created.

Streaming Tables, Materialized Views



Steps for the Practical Demo of DLT Pipeline Execution :

Step 1 - Load the Data from the Landing Zone and move it to the Bronze Layer.

Step 2 - Take the Data from the Bronze layer, clean the data and load it to the Silver Cleaned.

Step 3 - Take the Customers Data from the Silver Cleaned layer, handle the Duplicates using SCD2 and load it to the Silver Layer.

Step 4 - Take the Orders Data from the Silver Cleaned layer, handle the Duplicates using simple Merge and load it to the Silver Layer.

Step 5 - Take the data from the Silver Layer, perform Aggregations and load it to the Gold Layer.

The Core Abstractions of DLT :

- **Streaming Table** is a Delta table that has a Stream writing to it. This is used for ingesting huge amounts of data where recomputation is not affordable.
- **Materialized View** are the results of the query stored in the Delta Table.

Expectations :

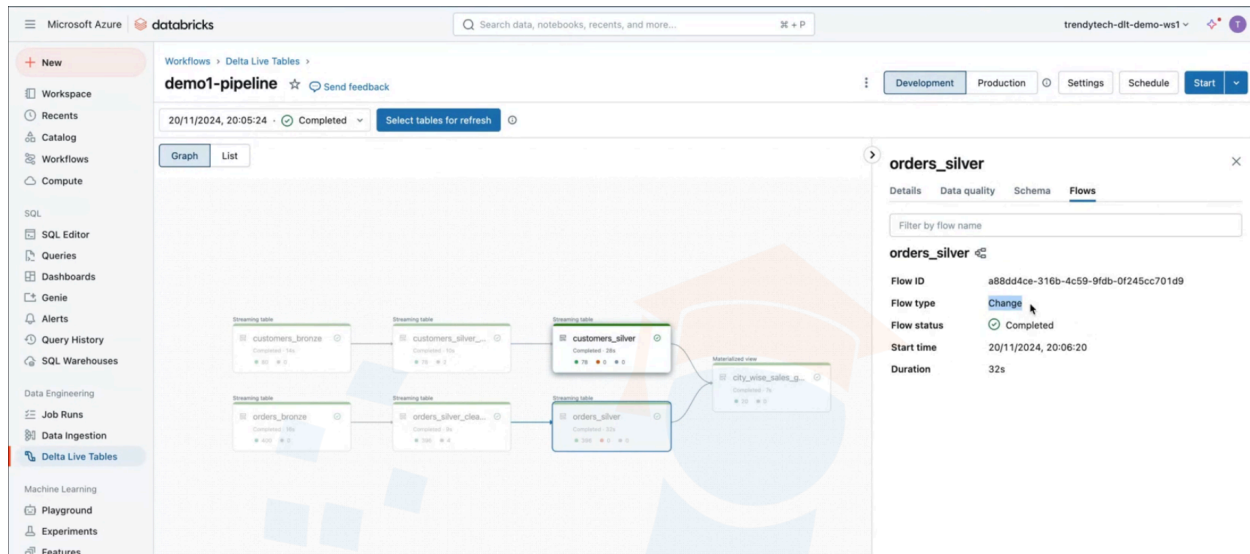
- Do not drop a table. Just raise a warning.
- Drop bad records
- Abort processing for a single bad record

Create a Pipeline under the Delta Live Table

- Give a pipeline name.
- Select the Pipeline mode (preferred Trigger as Continuous will be charge more)
- Select the Storage option (Hive metastore / Unity Catalog)
- Give the Target Schema Name
- Provide the Storage Location
- Select the Cluster Compute configuration (Cluster Policy : None, Cluster Mode : Fixed Size, Workers : 0 [only when it is Hive Metastore], Worker

Type : Standard_DS3_V2) -> Then Create the Pipeline after setting all the configurations.

- Then choose - “Start” option to run the Pipeline.



Note :

- Hive metastore storage is local to the specific workspace.
- The Unity Catalog Storage is a centralized repository which can be shared across workspaces.
- If you need to upload new incoming records and do an incremental load, then you will just need to “**Start**” the pipeline again which will pick only the new records or updated records. In case there is a need for “full load” that is all the records from the beginning needs to be added, then select “**Full Refresh All**” option under Start.
- For Materialized view, we have something called **Enzyme** directory (`_enzyme_log`). Enzyme is a set of Optimizations, to ensure that the data loading happens in the most efficient way. Wherever required partial recomputation is carried out rather than performing the complete computation all over again and still achieve the right results.
- Unity Catalog is Preferred over Hive Catalog.

External Locations and Modern Implementation



Leveraging Unity Catalog for Data Pipeline Implementation

Implementation of Data Pipeline using the current approach :

- Earlier Hive Metastore was used. Now, Unity Catalog is used.
- Earlier Mount Points we used to connect to the Storage which is not a secured approach. Now, External Locations are used which are more secured and governed by Unity Catalog.

Converting or Making changes in the existing code to incorporate the above mentioned changes -

1. Wherever, 'mnt' points are used, they are replaced with the external location.

Steps for External Storage creation -

- a. Create a container under blob storage
- b. Create an Input Folder within the container.
- c. Create Separate folders for different Datasets.
- d. Set External Location for the Input folder path. (Databricks Workspace -> Catalog -> External Location (Fill in all the fields - Storage Location URL & Storage Credentials) -> Create. Copy the path after testing the external location connection.
- e. This URL is then used in the Notebook, in-place of **mnt points**, as shown in the diagram below -



```
CREATE STREAMING TABLE orders_bronze
as
SELECT *,
input_file_name() as filename,
current_timestamp() as load_time
FROM cloud_files('abfss://retail@ttmstorageaccount001.dfs.core.windows.net/input/orders',

CREATE STREAMING TABLE customers_bronze
as
SELECT *,
input_file_name() as filename,
current_timestamp() as load_time
FROM cloud_files('abfss://retail@ttmstorageaccount001.dfs.core.windows.net/input/customers'
```

2. **input_file_name()** function works only with Hive Metastore. However, when using Unity Catalog, it has to be replaced with **_metadata.file_name** to capture the File.
3. **Live keyword** is not required as the complete path is defined, Catalog name and Schema names are specified while pipeline creation.

4. **Apply Changes APIs** are recommended to be replaced with **Auto CDC APIs**. However, the applychanges APIs are still available.

```
APPLY CHANGES into customers_silver
from STREAM(customers_silver_cleaned)
keys(customer_id)
sequence by load_time
stored as scd type 2;
```

```
CREATE STREAMING TABLE orders_silver;

CREATE FLOW orders_silver_flow AS AUTO CDC
into orders_silver
from STREAM(orders_silver_cleaned)
keys(order_id)
sequence by load_time;
```

5. While creating the Pipeline, we can choose any of the storage options like the **Hive Metastore** or **Unity Catalog**. However, the moment serverless is chosen under the General section, the only storage option that would be available is Unity Catalog. Future of Databricks will be Unity Catalog | Serverless and so on.
- Define the Catalog and the Schema
6. Once the pipeline is created, the resource will be allocated instantly as serverless is used. And the Pipeline can be Run.
7. **Lineage** (upstream / downstream) can be checked under **Catalog -> Select the Table -> Lineage Tab** .

DLT Demo



Important Note :

- Hive Metastore to be replaced with Unity Catalog
- In place of Classic Compute, Serverless needs to be used.
- Instead of Mount Points, Volumes / External Location can be used.
- Live Keyword is not required any longer. Instead use a 3 level Namespace.

Quick Demos :

Demo 1 Scenario - Load the data from the public path and bring it to Volumes. Create a Volume, Download file from the internet and add this file from the public path to the Volume.

Catalog

Schema

Tables

Views

Volumes (Files)

(Table is to hold the data in a tabular Structure. Volumes are used to hold the Raw Files)

Find Trending names in 2021 -

```
my_catalog = "trendytech-dev"
my_schema = "demo3_db"
my_volume = "demo3_volume"

spark.sql(f"CREATE SCHEMA IF NOT EXISTS {my_catalog}.{my_schema}")
spark.sql(f"CREATE VOLUME IF NOT EXISTS {my_catalog}.{my_schema}.{my_volume}")

volume_path = f"/Volumes/{my_catalog}/{my_schema}/{my_volume}/"
download_url = "https://health.data.ny.gov/api/views/jxy9-yhdk/rows.csv"
filename = "babynames.csv"

dbutils.fs.cp(download_url, volume_path + filename)
```

```
dbutils.fs.cp("https://health.data.ny.gov/api/views/jxy9-yhdk/rows.csv", "/Volumes/trendytech-dev/demo3_db/demo3_volume/baby_names.csv")
```

List down the following 3 names

1. my_catalog - Catalog name
2. my_schema - Schema name
3. my_volume - Volume name

Provide the above as advanced configurations as Key-Value pairs during Pipeline creation.

Demo 2 Scenario -

Selecting more than one Notebook as a Code for execution. Modularizing the code and segregating the code into multiple notebooks for better readability.

How to run multiple Notebooks and handle dependencies?

Go to **Delta Live Tables** -> Create Pipeline -> Under Path (Select the all the Notebooks required) Dependencies will be automatically handled -> Choose the Storage Location -> Create Pipeline -> Start Pipeline

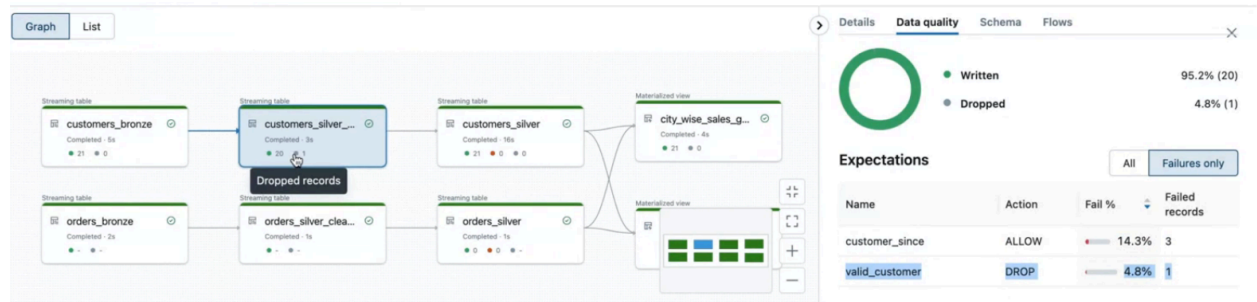
Note : This will be important when you have a very complicated code and needs to be modularized by splitting the code into multiple Notebooks for better understanding. However, the dependencies will still be handled internally by the framework.

Demo 3 Scenario -

Applying the Constraints and checking the execution results. As demonstrated in the code below -

```
CREATE STREAMING TABLE orders_silver_cleaned(  
  constraint valid_order EXPECT (order_id is NOT NULL) ON VIOLATION DROP ROW,  
  constraint valid_customer EXPECT (customer_id is NOT NULL) ON VIOLATION DROP ROW,  
  constraint valid_amount EXPECT (total_amount > 0) ON VIOLATION FAIL UPDATE  
) AS SELECT orderid AS order_id, orderdate as order_date, customerid as customer_id, totalamount as total_amount, status,  
  filename as file_name,  
  load time  
FROM STREAM(live.orders_bronze);
```

While uploading the second dataset, some of the entries had customer-id with null value and the registration date as null. Need to check how the violations are handled?



The entries which violated the constraints were dropped.
Update fails when constraints are not met.

Note :

Enzyme Optimization in Materialized View -

1. Monotonic Append
2. Partition Recompute
3. Merge Updates
4. Full Compute

(For DLT serverless, Enzymes is enabled by default)

TRENDY TECH
UPLIFT YOUR CAREER!

ETL Pipeline



Understanding the New IDE that helps in the development more quickly

Considering the previous **dlt-demo-2** code and converting it to be executed in the new IDE.

Important changes that were done -

1. External Locations in place of mount points
2. Unity catalog in place of Hive Metastore
3. Live Keyword no longer used.
4. Auto CDC in place of Apply Changes APIs
CREATE FLOW orders_silver_flow AS AUTO CDC
5. Whenever possible use Serverless.

Development on NEW IDE

New -> ETL Pipeline

OR

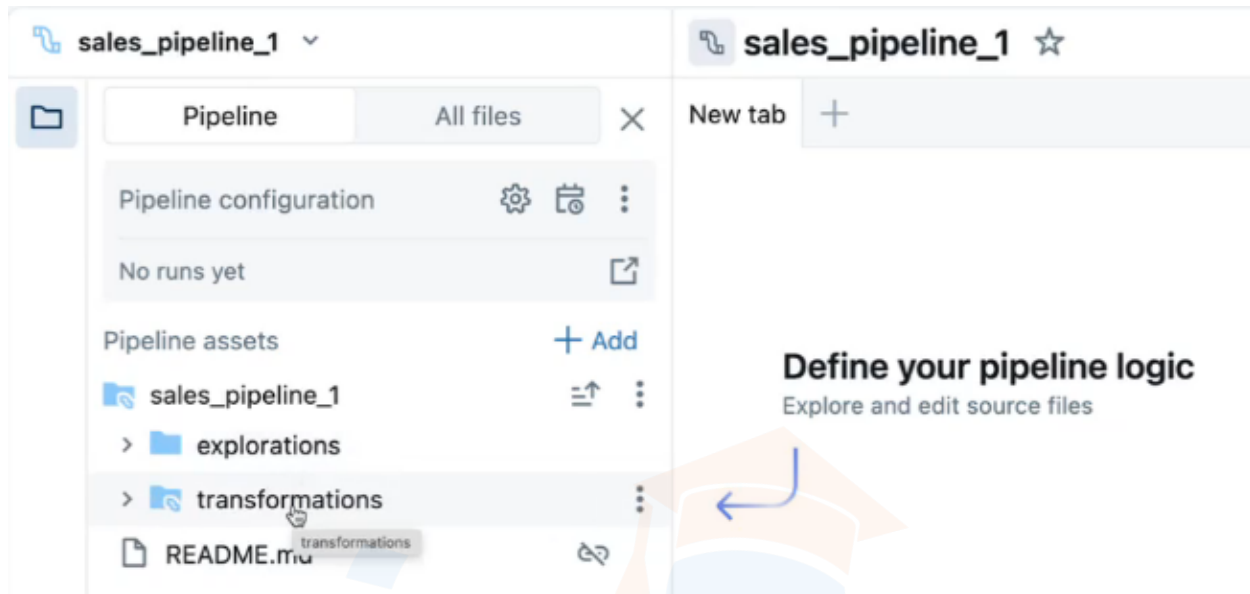
Jobs & Pipeline -> ETL Pipeline

OR

Job Runs -> ETL Pipeline

Note : Enable the “Lakeflow Pipeline Editor : ON” if disabled.

- Give a Name to the Pipeline
- The name given to the Pipeline is the **Root Folder**, that consists of subfolders as shown in the diagram below



When sample SQL code option is selected, the framework gives a sample code with the Structure

Example Structure :

- Sales_pipeline_1 (Root Folder)
 - explorations
 - transformations

The Files in the explorations folder are for the adhoc experimental code analysis and are not executed when the pipeline runs.

The Files in the transformations folder is the actual code and will be executed as part of the pipeline.

We can perform the Dry Run to check the sample results.

The code that needs to be executed has to go under the transformations folder.